

Sprint 7 Structured Notes: Loops and Iteration

0. What Sprint 7 Is About

Sprint 7 is about **repeating work safely** in JavaScript.

In earlier sprints, you learned how to create values, store them in variables, write conditions, create functions, and store many values inside arrays. Sprint 7 teaches you how to go through those values one by one and run code repeatedly.

Main goal:

Repeat work safely and choose the right loop for the data.

By the end of Sprint 7, you should understand:

- what loops are
- why loops are useful
- standard `for` loops
- loop initialization, condition, and update
- loop counters
- array indexes inside loops
- `for...of`
- `for...in`
- `while`
- `do...while`
- infinite loops
- `break`
- `continue`
- nested loops
- looping through 2D arrays
- choosing the correct loop type
- common beginner mistakes with loops
- how loops connect to arrays, strings, objects, functions, and conditions

1. Current Progress Before Sprint 7

Sprint 7 depends on the earlier sprints.

Earlier Sprint	What You Learned	How It Helps With Sprint 7
Sprint 1	JavaScript can respond to actions on a web page.	Loops can repeat behavior when page data changes.
Sprint 2	Values, data types, <code>let</code> , <code>const</code> , and references.	Loops repeatedly work with stored values.
Sprint 3	Strings, indexes, <code>.length</code> , <code>includes()</code> , <code>indexOf()</code> , and <code>slice()</code> .	Strings can be looped through because they are index-based and iterable.
Sprint 4	Numbers, booleans, comparisons, <code>if</code> , <code>else</code> , and logical operators.	Loops use boolean conditions to decide whether to continue.
Sprint 5	Functions, parameters, return values, and scope.	Loop logic can be placed inside reusable functions.
Sprint 6	Arrays, indexes, <code>.length</code> , <code>push()</code> , <code>splice()</code> , shallow copies, and 2D arrays.	Sprint 7 teaches how to process array values repeatedly.

Simple summary:

Sprint 6 taught you how to store many values. Sprint 7 teaches you how to move through those values and do work with them.

2. The Core Idea of Loops

A **loop** repeats a block of code.

Without a loop, repeated work looks like this:

```
const scores = [80, 95, 72, 100];

console.log(scores[0]);
console.log(scores[1]);
console.log(scores[2]);
console.log(scores[3]);
```

Expected output:

```
80
95
```

```
72  
100
```

This works, but it is repetitive.

A loop makes the code cleaner:

```
const scores = [80, 95, 72, 100];  
  
for (const score of scores) {  
  console.log(score);  
}
```

Expected output:

```
80  
95  
72  
100
```

Simple meaning:

For each score in the scores array, print the score.

Loops are useful when you need to:

- print many values
- calculate totals
- search for an item
- count matching values
- validate many values
- update many values
- read every row and column in a grid
- process arrays, strings, and objects

3. Standard `for` Loops

A standard `for` loop is useful when you need index control.

Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Expected output:

```
0  
1  
2  
3  
4
```

A standard `for` loop has three parts:

```
for (initialization; condition; update) {  
  // repeated code  
}
```

Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Part	Code	Simple Meaning
Initialization	<code>let i = 0</code>	Start the counter at 0.
Condition	<code>i < 5</code>	Keep looping while this is true.
Update	<code>i++</code>	Add 1 after each loop cycle.

4. What `i` Means

In many beginner loops, you will see the variable `i`.

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

`i` is a common name for an index or counter.

Simple meaning:

`i` keeps track of which loop step you are currently on.

It starts at `0` because arrays and strings are zero-indexed.

5. How a `for` Loop Runs Step by Step

Code:

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

Dry run:

Step	<code>i</code> Value	Condition <code>i < 3</code>	What Happens
1	<code>0</code>	<code>true</code>	Print <code>0</code> , then <code>i++</code> makes <code>i</code> become <code>1</code> .
2	<code>1</code>	<code>true</code>	Print <code>1</code> , then <code>i++</code> makes <code>i</code> become <code>2</code> .
3	<code>2</code>	<code>true</code>	Print <code>2</code> , then <code>i++</code> makes <code>i</code> become <code>3</code> .
4	<code>3</code>	<code>false</code>	Stop the loop.

Expected output:

```
0  
1  
2
```

Important:

```
i < 3
```

means the loop stops before `i` reaches `3`.

6. Looping Through an Array With a Standard `for` Loop

Use a standard `for` loop when you need the index.

```
const fruits = ["apple", "banana", "cherry"];

for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}
```

Expected output:

```
0 apple
1 banana
2 cherry
```

What this code gives you:

```
i          // the index
fruits[i]  // the value at that index
```

Dry run:

i	fruits[i]
0	"apple"
1	"banana"
2	"cherry"

The condition is:

```
i < fruits.length
```

If `fruits.length` is `3`, the loop runs while `i` is `0`, `1`, and `2`.

That matches the valid array indexes.

7. Why Loops Often Start at 0

Arrays start at index `0`.

```
const fruits = ["apple", "banana", "cherry"];
```

Indexes:

Index	Value
0	"apple"
1	"banana"
2	"cherry"

So this loop starts at `0`:

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

Beginner rule:

When looping through arrays with indexes, start at `0` and continue while `i < array.length`.

8. Off-by-One Mistakes

An **off-by-one mistake** happens when your loop runs one too many times or one too few times.

Problem example:

```
const fruits = ["apple", "banana", "cherry"];  
  
for (let i = 0; i <= fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

Expected output:

```
apple  
banana  
cherry  
undefined
```

Why does `undefined` appear?

`fruits.length` is `3`, but the last valid index is `2`.

This condition is wrong here:

```
i <= fruits.length
```

Correct version:

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

Beginner rule:

For array indexes, use `< array.length`, not `<= array.length`.

9. `for...of`

Use `for...of` when you want the values directly.

```
const grades = [90, 82, 77];  
  
for (const grade of grades) {  
  console.log(grade);  
}
```

Expected output:

```
90  
82  
77
```


Read this as:

For each grade inside grades, run this code.

A `for...of` loop is clean when you do not need the index.

Example:

```
const names = ["Ada", "Grace", "Linus"];

for (const name of names) {
  console.log(`Hello, ${name}`);
}
```

Expected output:

```
Hello, Ada
Hello, Grace
Hello, Linus
```

10. `for...of` With Strings

Strings are also iterable, so `for...of` can loop through characters.

```
const word = "code";

for (const letter of word) {
  console.log(letter);
}
```

Expected output:

```
c
o
d
e
```

Simple meaning:

For each character in the string, run the code.

This is useful for simple text-processing tasks.

11. When to Use `for...of`

Use `for...of` when:

- you are looping through an array
- you are looping through a string
- you only need the value
- you do not need the index
- you want cleaner, easier-to-read code

Example:

```
const tasks = ["read", "code", "review"];

for (const task of tasks) {
  console.log(task);
}
```

Expected output:

```
read
code
review
```

Beginner rule:

Use `for...of` for arrays and strings when you only need values.

12. Standard `for` Loop vs `for...of`

Both can loop through arrays.

Standard `for` loop:

```
const colors = ["red", "green", "blue"];

for (let i = 0; i < colors.length; i++) {
```

```
    console.log(i, colors[i]);  
  }
```

Expected output:

```
0 red  
1 green  
2 blue
```

`for...of` loop:

```
const colors = ["red", "green", "blue"];  
  
for (const color of colors) {  
  console.log(color);  
}
```

Expected output:

```
red  
green  
blue
```

Comparison:

Situation	Better Choice
You need the index.	Standard <code>for</code> loop
You only need the value.	<code>for...of</code>
You need to update by position.	Standard <code>for</code> loop
You are just reading each value.	<code>for...of</code>

13. `for...in`

Use `for...in` to loop through object keys.

Example:

```
const user = {  
  name: "Ada",  
  active: true,  
  score: 95  
};  
  
for (const key in user) {  
  console.log(key);  
}
```

Expected output:

```
name  
active  
score
```

Simple meaning:

For each key in the user object, run this code.

14. Getting Object Values With `for...in`

`for...in` gives you keys.

To get the value for each key, use bracket notation.

```
const user = {  
  name: "Ada",  
  active: true,  
  score: 95  
};  
  
for (const key in user) {  
  console.log(key, user[key]);  
}
```

Expected output:

```
name Ada
active true
score 95
```

Why bracket notation?

```
user[key]
```

This means:

Use the current value of `key` to access the matching property.

Dry run:

key	user[key]
"name"	"Ada"
"active"	true
"score"	95

15. `for...of` vs `for...in`

These two loops are different.

Loop	Main Use	What It Gives You
<code>for...of</code>	Arrays and strings	Values
<code>for...in</code>	Objects	Keys

Example with `for...of`:

```
const fruits = ["apple", "banana"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

Expected output:

```
apple  
banana
```

Example with `for...in`:

```
const user = {  
  name: "Ada",  
  active: true  
};  
  
for (const key in user) {  
  console.log(key, user[key]);  
}
```

Expected output:

```
name Ada  
active true
```

Beginner rule:

Use `for...of` for array/string values. Use `for...in` for object keys.

16. Avoid Using `for...in` as Your Default Array Loop

This works technically:

```
const fruits = ["apple", "banana", "cherry"];  
  
for (const index in fruits) {  
  console.log(index, fruits[index]);  
}
```

Possible output:

```
0 apple  
1 banana  
2 cherry
```

But for beginner array code, this is not the best default.

Why?

Because `for...in` gives keys, not direct values. For arrays, those keys are index-like strings.

Cleaner array value loop:

```
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

Cleaner array index loop:

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(i, fruits[i]);  
}
```

Beginner rule:

Do not use `for...in` as your normal array loop. Use `for...of` or a standard `for` loop instead.

17. `while` Loops

A `while` loop runs while a condition is true.

```
let count = 3;  
  
while (count > 0) {  
  console.log(count);  
  count--;  
}
```

Expected output:

```
3  
2  
1
```

Read this as:

While `count` is greater than `0`, print count and decrease it.

18. How a `while` Loop Works

A `while` loop has this structure:

```
while (condition) {  
  // repeated code  
}
```

The condition is checked before each loop cycle.

Example:

```
let count = 3;  
  
while (count > 0) {  
  console.log(count);  
  count--;  
}
```

Dry run:

Step	count	Condition	count > 0	What Happens
1	3	true		Print 3, then decrease to 2.
2	2	true		Print 2, then decrease to 1.
3	1	true		Print 1, then decrease to 0.
4	0	false		Stop.

19. When to Use `while`

Use `while` when you do not know exactly how many times the loop should run, but you know the condition for stopping.

Example:


```
let energy = 5;

while (energy > 0) {
  console.log("Keep working");
  energy--;
}
```

Expected output:

```
Keep working
Keep working
Keep working
Keep working
Keep working
```

Use `while` for situations like:

- repeat until a value reaches a limit
- repeat while something is still true
- keep checking until a condition changes
- process data when the number of steps is not fixed in advance

20. Infinite Loops

An infinite loop happens when a loop never stops.

Broken example:

```
let count = 3;

while (count > 0) {
  console.log(count);
}
```

This loop is dangerous because `count` never changes.

The condition stays true forever:

```
count > 0
```

Since `count` stays `3`, the condition is always true.

Correct version:

```
let count = 3;

while (count > 0) {
  console.log(count);
  count--;
}
```

Now `count` changes each time.

Eventually, `count` becomes `0`, and the loop stops.

Beginner rule:

In a `while` loop, always make sure the value used in the condition changes.

21. Infinite Loop Safety Checklist

Before running a `while` loop, ask:

1. What condition controls the loop?
2. Which variable does the condition depend on?
3. Does that variable change inside the loop?
4. Will the condition eventually become false?
5. Is there a clear stopping point?

Example safe loop:

```
let attempts = 0;

while (attempts < 3) {
  console.log("Try again");
  attempts++;
}
```

Why it is safe:

- the condition depends on `attempts`
 - `attempts` increases inside the loop
 - eventually `attempts < 3` becomes false
-

22. do...while Loops

A `do...while` loop runs the body first, then checks the condition.

Example:

```
let count = 0;

do {
  console.log("Runs once");
  count++;
} while (count < 1);
```

Expected output:

```
Runs once
```

Structure:

```
do {
  // repeated code
} while (condition);
```

Important:

A `do...while` loop always runs at least once.

23. while vs do...while

A `while` loop checks first.

A `do...while` loop runs first.

Example:

```
let number = 10;

while (number < 5) {
  console.log("while ran");
}
```

```
do {  
  console.log("do...while ran");  
} while (number < 5);
```

Expected output:

```
do...while ran
```

Why?

The `while` loop checks this first:

```
number < 5
```

Since `10 < 5` is false, the `while` loop does not run.

The `do...while` loop runs once before checking the condition.

Comparison:

Loop	Checks Condition	Can Run Zero Times?	Always Runs At Least Once?
<code>while</code>	Before the body	Yes	No
<code>do...while</code>	After the body	No	Yes

24. `break`

`break` exits a loop early.

Example:

```
const scores = [40, 55, 72, 90];  
  
for (const score of scores) {  
  if (score >= 70) {  
    console.log(`First passing score: ${score}`);  
    break;  
  }  
}
```

```
}  
}
```

Expected output:

First passing score: 72

Without `break`, the loop would continue checking `90`.

With `break`, the loop stops as soon as the first passing score is found.

Use `break` when:

- you found what you were searching for
- you do not need to continue the loop
- continuing would waste work
- the answer is already known

25. `break` Dry Run

Code:

```
const numbers = [3, 7, 10, 12];  
  
for (const number of numbers) {  
  if (number % 2 === 0) {  
    console.log(number);  
    break;  
  }  
}
```

Dry run:

Number	Is Even?	What Happens?
3	No	Continue.
7	No	Continue.
10	Yes	Print <code>10</code> , then <code>break</code> stops the loop.
12	Not checked	Loop already stopped.

Expected output:

10

26. `continue`

`continue` skips the current loop step and moves to the next one.

Example:

```
const scores = [80, 95, 72, 100];
let total = 0;

for (const score of scores) {
  if (score < 75) {
    continue;
  }

  total += score;
}

console.log(total);
```

Expected output:

275

Step-by-step:

Score	What Happens
80	Added to total.
95	Added to total.
72	Skipped because it is below 75.
100	Added to total.

Calculation:

$80 + 95 + 100 = 275$

Use `continue` when:

- one item should be skipped
- the loop should keep running
- you want to ignore invalid or unwanted values

27. `break` vs `continue`

`break` and `continue` both control loop flow, but they do different things.

Keyword	Meaning	What Happens Next?
<code>break</code>	Stop the whole loop.	JavaScript exits the loop.
<code>continue</code>	Skip this one loop step.	JavaScript moves to the next iteration.

Example with `break`:

```
const numbers = [1, 2, 3, 4];

for (const number of numbers) {
  if (number === 3) {
    break;
  }
  console.log(number);
}
```

Expected output:

```
1
2
```

The loop stops completely when `number` is `3`.

Example with `continue`:

```
const numbers = [1, 2, 3, 4];

for (const number of numbers) {
  if (number === 3) {
    continue;
  }
}
```

```
    console.log(number);  
  }
```

Expected output:

```
1  
2  
4
```

The loop skips `3`, but continues to `4`.

28. Conditions Inside Loops

Loops often use `if` statements inside them.

Example:

```
const scores = [90, 45, 80, 60];  
  
for (const score of scores) {  
  if (score >= 70) {  
    console.log(`${score} is passing`);  
  } else {  
    console.log(`${score} is not passing`);  
  }  
}
```

Expected output:

```
90 is passing  
45 is not passing  
80 is passing  
60 is not passing
```

Simple meaning:

The loop visits each score. The condition decides what to do with that score.

This pattern is very common:


```
for (const item of items) {  
  if (condition) {  
    // do something  
  }  
}
```

29. Accumulators in Loops

An accumulator is a variable that collects a result while a loop runs.

Example: calculating a total.

```
const prices = [10, 20, 30];  
let total = 0;  
  
for (const price of prices) {  
  total += price;  
}  
  
console.log(total);
```

Expected output:

60

Dry run:

Price	Total Before	Total After
10	0	10
20	10	30
30	30	60

Common accumulator patterns:

```
let total = 0;      // for sums  
let count = 0;     // for counting  
let result = "";   // for building strings  
let found = false; // for tracking whether something exists
```

30. Counting With Loops

Example: count scores that are passing.

```
const scores = [90, 45, 80, 60, 100];
let passingCount = 0;

for (const score of scores) {
  if (score >= 70) {
    passingCount++;
  }
}

console.log(passingCount);
```

Expected output:

3

Passing scores:

90
80
100

So the count is 3.

31. Searching With Loops

Example: check whether an array contains a specific name.

```
const names = ["Ada", "Grace", "Linus"];
let found = false;

for (const name of names) {
  if (name === "Grace") {
    found = true;
    break;
  }
}
```

```
console.log(found);
```

Expected output:

```
true
```

Why use `break`?

Because once `"Grace"` is found, there is no need to keep searching.

32. Building a New Array With a Loop

You can use a loop to create a new array.

Example: keep only passing scores.

```
const scores = [90, 45, 80, 60, 100];
const passingScores = [];

for (const score of scores) {
  if (score >= 70) {
    passingScores.push(score);
  }
}

console.log(passingScores);
```

Expected output:

```
[90, 80, 100]
```

Simple meaning:

Start with an empty array. Add only the values that match the condition.

33. Nested Loops

A nested loop is a loop inside another loop.

Example:

```
for (let outer = 1; outer <= 2; outer++) {  
  for (let inner = 1; inner <= 3; inner++) {  
    console.log(outer, inner);  
  }  
}
```

Expected output:

```
1 1  
1 2  
1 3  
2 1  
2 2  
2 3
```

Simple meaning:

For each outer loop step, the inner loop runs completely.

Dry run:

Outer	Inner Values Printed
1	1, 2, 3
2	1, 2, 3

34. Nested Loops With 2D Arrays

A 2D array is an array of arrays.

Example:

```
const grid = [  
  ["A1", "A2"],  
  ["B1", "B2"]  
];
```

Think of it like this:

Row	Column 0	Column 1
0	A1	A2
1	B1	B2

To loop through every value, use a nested loop.

```
const grid = [  
  ["A1", "A2"],  
  ["B1", "B2"]  
];  
  
for (let row = 0; row < grid.length; row++) {  
  for (let col = 0; col < grid[row].length; col++) {  
    console.log(row, col, grid[row][col]);  
  }  
}
```

Expected output:

```
0 0 A1  
0 1 A2  
1 0 B1  
1 1 B2
```

Read this:

```
grid[row][col]
```

as:

Go to the row, then go to the column inside that row.

35. Nested Loops Can Become Slow

Nested loops are useful, but they can do a lot of work.

Example:

```
for (const a of listA) {  
  for (const b of listB) {
```

```
    console.log(a, b);  
  }  
}
```

If `listA` has `3` items and `listB` has `4` items, the inner code runs:

```
3 * 4 = 12
```

If both lists have `1000` items, the inner code runs:

```
1000 * 1000 = 1,000,000
```

Beginner rule:

Nested loops are normal for grids and 2D arrays, but avoid unnecessary nested loops for large data.

36. Loop Choice Guide

Use this guide when choosing a loop.

Situation	Best Choice
You need index numbers.	Standard <code>for</code> loop
You need array values only.	<code>for...of</code>
You need string characters.	<code>for...of</code>
You need object keys.	<code>for...in</code>
You need object keys and values.	<code>for...in</code> with bracket notation
You need to repeat while a condition is true.	<code>while</code>
The loop must run at least once.	<code>do...while</code>
You found the answer and want to stop.	<code>break</code>
You want to skip one item and keep going.	<code>continue</code>
You need rows and columns.	Nested loops

37. Core Sprint 7 Code Pattern

Type this code yourself.

Before running it, predict the output.

```
const grades = [90, 82, 77];

for (const grade of grades) {
  console.log(grade);
}

const user = { name: "Ada", active: true };

for (const key in user) {
  console.log(key, user[key]);
}

let count = 3;

while (count > 0) {
  console.log(count);
  count--;
}
```

Expected output:

```
90
82
77
name Ada
active true
3
2
1
```

What this teaches:

1. `for...of` loops through array values.
2. `for...in` loops through object keys.
3. Bracket notation gets object values from dynamic keys.
4. `while` repeats while a condition is true.
5. The `while` loop must update the condition variable.

38. Mini-Build 1: Count Even Numbers

Task:

Count how many numbers in an array are even.

Code:

```
function countEvenNumbers(numbers) {  
  let count = 0;  
  
  for (const number of numbers) {  
    if (number % 2 === 0) {  
      count++;  
    }  
  }  
  
  return count;  
}  
  
console.log(countEvenNumbers([1, 2, 3, 4, 5, 6]));
```

Expected output:

3

Why?

Even numbers are:

2
4
6

So the count is:

3

Concepts used:

- function
- parameter

- `let`
- accumulator variable
- `for...of`
- `if`
- remainder operator `%`
- strict equality `===`
- increment `count++`
- `return`

39. Mini-Build 2: Sum Scores 75 and Above

Task:

Add only scores that are `75` or higher.

Code:

```
function sumStrongScores(scores) {  
  let total = 0;  
  
  for (const score of scores) {  
    if (score < 75) {  
      continue;  
    }  
  
    total += score;  
  }  
  
  return total;  
}  
  
console.log(sumStrongScores([80, 95, 72, 100]));
```

Expected output:

275

Why?

`72` is skipped.

Calculation:

80 + 95 + 100 = 275

Concepts used:

- function
- array parameter
- accumulator variable
- `for...of`
- `if`
- `continue`
- compound assignment `+=`
- `return`

40. Mini-Build 3: Find First Passing Score

Task:

Find the first score that is `70` or higher.

Code:

```
function findFirstPassingScore(scores) {  
  for (const score of scores) {  
    if (score >= 70) {  
      return score;  
    }  
  }  
  
  return null;  
}  
  
console.log(findFirstPassingScore([40, 55, 72, 90]));  
console.log(findFirstPassingScore([40, 55, 60]));
```

Expected output:

```
72  
null
```

Important:

Inside a function, `return` can stop the function early.

This is similar to `break`, but it also gives a value back.

41. Mini-Build 4: Print a Grid

Task:

Print every value in a 2D array.

Code:

```
function printGrid(grid) {  
  for (let row = 0; row < grid.length; row++) {  
    for (let col = 0; col < grid[row].length; col++) {  
      console.log(grid[row][col]);  
    }  
  }  
}  
  
printGrid([  
  ["A1", "A2"],  
  ["B1", "B2"],  
  ["C1", "C2"]  
]);
```

Expected output:

```
A1  
A2  
B1  
B2  
C1  
C2
```

Concepts used:

- function
 - 2D array
 - nested loops
 - row index
 - column index
 - `grid[row][col]`
-

42. Practice Drill 1: Print Indexes

Code:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Expected output:

```
0  
1  
2  
3  
4
```

What to notice:

- loop starts at `0`
 - loop stops before `5`
 - `i++` updates the counter
-

43. Practice Drill 2: Print Array Values

Code:

```
const colors = ["red", "green", "blue"];  
  
for (const color of colors) {  
  console.log(color);  
}
```

Expected output:

```
red  
green  
blue
```

What to notice:

- `for...of` gives direct values
 - no index is needed
-

44. Practice Drill 3: Print Object Keys and Values

Code:

```
const course = {  
  name: "JavaScript",  
  sprint: 7,  
  topic: "Loops"  
};  
  
for (const key in course) {  
  console.log(key, course[key]);  
}
```

Expected output:

```
name JavaScript  
sprint 7  
topic Loops
```

What to notice:

- `for...in` gives keys
 - `course[key]` gets the value
-

45. Practice Drill 4: Fix the Infinite Loop

Broken code:

```
let count = 3;  
  
while (count > 0) {  
  console.log(count);  
}
```

Problem:

`count` never changes, so the loop never stops.

Fixed code:

```
let count = 3;

while (count > 0) {
  console.log(count);
  count--;
}
```

Expected output:

```
3
2
1
```

46. Practice Drill 5: Skip Small Numbers

Task:

Print only numbers that are `5` or greater.

Code:

```
const numbers = [2, 5, 8, 1, 10];

for (const number of numbers) {
  if (number < 5) {
    continue;
  }

  console.log(number);
}
```

Expected output:

5
8
10

What to notice:

- 2 is skipped
- 1 is skipped
- the loop continues after skipped values

47. Practice Drill 6: Stop When You Find a Match

Task:

Stop when the value "target" is found.

Code:

```
const words = ["start", "middle", "target", "end"];

for (const word of words) {
  if (word === "target") {
    console.log("Found target");
    break;
  }

  console.log(word);
}
```

Expected output:

```
start
middle
Found target
```

What to notice:

- "end" is not printed
- break stops the loop

48. Common Beginner Mistakes

Mistake 1: Using `<= array.length`

Wrong:

```
for (let i = 0; i <= items.length; i++) {  
  console.log(items[i]);  
}
```

Why it is wrong:

It tries to access one index past the end.

Correct:

```
for (let i = 0; i < items.length; i++) {  
  console.log(items[i]);  
}
```

Mistake 2: Forgetting to Update a `while` Loop

Wrong:

```
let count = 5;  
  
while (count > 0) {  
  console.log(count);  
}
```

Correct:

```
let count = 5;  
  
while (count > 0) {  
  console.log(count);  
  count--;  
}
```


Mistake 3: Confusing `for...of` and `for...in`

Wrong for normal array values:

```
const fruits = ["apple", "banana"];

for (const fruit in fruits) {
  console.log(fruit);
}
```

Possible output:

```
0
1
```

Correct:

```
for (const fruit of fruits) {
  console.log(fruit);
}
```

Expected output:

```
apple
banana
```

Mistake 4: Forgetting Bracket Notation With Object Keys

Wrong:

```
const user = { name: "Ada" };

for (const key in user) {
  console.log(user.key);
}
```

Output:

undefined

Why?

`user.key` looks for a property literally named `key`.

Correct:

```
for (const key in user) {  
  console.log(user[key]);  
}
```

Expected output:

Ada

Mistake 5: Using `break` When You Meant `continue`

If you want to skip one item, use `continue`.

```
for (const number of numbers) {  
  if (number < 0) {  
    continue;  
  }  
  
  console.log(number);  
}
```

If you want to stop the whole loop, use `break`.

```
for (const number of numbers) {  
  if (number < 0) {  
    break;  
  }  
  
  console.log(number);  
}
```

49. Beginner Rules for Sprint 7

1. Use a standard `for` loop when you need indexes.
 2. Use `for...of` when you only need values from an array or string.
 3. Use `for...in` when you need object keys.
 4. Use `while` when a condition controls how long the loop runs.
 5. Use `do...while` when the code must run at least once.
 6. Use `break` to stop a loop early.
 7. Use `continue` to skip one loop step.
 8. Be careful with infinite loops.
 9. Use `< array.length`, not `<= array.length`, for normal array loops.
 10. Use nested loops for 2D arrays, grids, rows, and columns.
 11. Avoid unnecessary nested loops with large data.
 12. Always predict the output before running loop code.
-

50. Sprint 7 Revision Questions

Answer these without looking at the notes.

1. What is a loop?
 2. Why are loops useful?
 3. What are the three parts of a standard `for` loop?
 4. Why do many array loops start at `0`?
 5. What does `i++` do?
 6. Why should array loops usually use `i < array.length`?
 7. What does `for...of` give you?
 8. What does `for...in` give you?
 9. When should you use bracket notation inside a `for...in` loop?
 10. What is a `while` loop?
 11. Why can a `while` loop become infinite?
 12. How is `do...while` different from `while`?
 13. What does `break` do?
 14. What does `continue` do?
 15. What is a nested loop?
 16. Why can nested loops become slow with large data?
 17. How do you loop through every value in a 2D array?
 18. Which loop should you choose when you only need array values?
 19. Which loop should you choose when you need object keys?
 20. Which loop should you choose when you need index control?
-

51. Sprint 7 Mastery Checklist

You are ready to move on from Sprint 7 when you can do these things:

- ☐ Explain what a loop does in simple words.
- ☐ Write a standard `for` loop from memory.
- ☐ Explain initialization, condition, and update.
- ☐ Loop through an array using indexes.
- ☐ Loop through an array using `for...of`.
- ☐ Loop through a string using `for...of`.
- ☐ Loop through an object using `for...in`.
- ☐ Use bracket notation to read object values from dynamic keys.
- ☐ Write a safe `while` loop.
- ☐ Explain how infinite loops happen.
- ☐ Write a `do...while` loop.
- ☐ Use `break` correctly.
- ☐ Use `continue` correctly.
- ☐ Write a nested loop for a 2D array.
- ☐ Choose the right loop for the data.
- ☐ Predict loop output before running the code.

52. Final Sprint 7 Summary

Sprint 7 teaches you how to repeat work safely.

The most important idea is:

A loop lets JavaScript run the same code multiple times without you writing the same line again and again.

Main loop choices:

```
for           // use when you need index control
for...of      // use for values in arrays and strings
for...in      // use for keys in objects
while         // use while a condition is true
do...while    // use when the code must run at least once
```

Main loop control tools:

```
break         // stop the loop
continue      // skip this step and keep looping
```

Main beginner warning:

Every loop needs a clear stopping point. If the stopping condition never becomes false, the loop can become infinite.

Main practical target:

You should be able to look at data and choose the correct loop: index loop, value loop, object-key loop, condition-based loop, or nested loop.